

ENUM

1. QUÉ ES ENUM EN JAVA
 2. PARA QUÉ SE USA
 3. VENTAJAS FRENTE A ALTERNATIVAS ANTIGUAS
 4. RELACIÓN CON JAVA 21 Y VERSIONES ANTERIORES
 5. CARACTERÍSTICAS PRINCIPALES DE UN ENUM
 6. MÉTODOS PRINCIPALES DE ENUM
 - 6.1 values()
 - 6.2 valueOf(String name)
 - 6.3 name()
 - 6.4 ordinal()
 - 6.5 toString()
 - 6.6 compareTo(E o)
 - 6.7 getDeclaringClass()
 - 6.8 equals(Object other)
 - 6.9 Enum.valueOf(Class<T> enumClass, String name)
 7. BUENAS PRÁCTICAS AL USAR ENUM
 8. DESVENTAJAS O LIMITACIONES DE ENUM
 9. EJEMPLOS DE USO
 - 10. CONCLUSIÓN TEÓRICA**
 - 11. EJEMPLO EN ECLIPSE**
 12. POSIBLES DIFICULTADES
 13. COMPARACIÓN CON VERSIONES ANTERIORES
-

1. QUÉ ES ENUM EN JAVA

Un enum es un tipo especial de clase que representa un conjunto finito, fijo y conocido de constantes. Se usa cuando una variable solo puede tomar uno de varios valores concretos, por ejemplo:

- estado de un pedido: **PENDIENTE, ENVIADO, ENTREGADO**
- prioridad de una incidencia: BAJA, MEDIA, ALTA
- tipo de usuario: **ADMIN, EMPLEADO, CLIENTE**

En Java, enum no es simplemente una lista de etiquetas. Cada constante de un enumerado es en realidad una instancia del tipo enum, lo que permite añadir:

- atributos
- constructores
- métodos
- implementación de interfaces
- comportamiento específico por constante

Eso hace que enum sea mucho más potente que usar cadenas como "ALTA" o números como 1, 2, 3.

2. PARA QUÉ SE USA

enum se usa principalmente para:

- evitar valores inválidos
- mejorar la legibilidad del código
- dar seguridad de tipos
- centralizar valores constantes relacionados
- asociar comportamiento a cada opción
- facilitar validaciones y menús

Ejemplo simple:

```
public enum Prioridad (BAJA, MEDIA, ALTA)
```

Con esto, una variable de tipo Prioridad solo podrá contener uno de esos tres valores.

3. VENTAJAS FRENTE A ALTERNATIVAS ANTIGUAS

Antes de usar enum, era muy habitual representar estados o tipos mediante:

```
public static final int  
    constantes String
```

Ejemplo antiguo:

```
public class EstadoPedido {  
    public static final int PENDIENTE = 1;  
    public static final int ENVIADO = 2;  
    public static final int ENTREGADO = 3;
```

Desventajas de ese enfoque antiguo:

- se pueden asignar valores incorrectos fácilmente
- los enteros no son expresivos
- los String pueden contener errores tipográficos
- no agrupan comportamiento
- complican el mantenimiento

Ventajas de enum:

- más seguro: solo acepta valores definidos
- más claro: el código se entiende mejor
- más mantenible
- permite métodos y atributos
- funciona muy bien con switch
- facilita recorridos con values()
- integra comportamiento orientado a objetos

4. RELACIÓN CON JAVA 21 Y VERSIONES ANTERIORES

Los enum existen desde Java 5, pero siguen plenamente vigentes en Java 21. En versiones modernas encajan aún mejor con características actuales del lenguaje, por ejemplo:

- switch mejorado
- expresiones switch
- patrones de código más legibles
- mejor integración con APIs modernas

Además, la clase base de todos los enumerados es `java.lang.Enum`, que define varios métodos importantes como `name()`, `ordinal()`, `compareTo()`, `getDeclaringClass()` y `valueOf()`.

5. CARACTERÍSTICAS PRINCIPALES DE UN ENUM

Un enum en Java:

- define un conjunto fijo de constantes
- extiende implícitamente de Enum
- no puede extender otra clase
- sí puede implementar interfaces
- puede tener atributos y métodos
- puede tener constructores privados
- puede declararse en fichero propio o dentro de una clase

- Ejemplo más completo:

```
public enum Prioridad {  
    BAJA(1, "Baja"),  
    NEDIA(2, "Media"),  
    ALTA(3, "Alta"),  
    URGENTE(4, "Urgente");  
  
    private final int nivel;  
    private final String descripcion;  
  
    Prioridad(int nivel, String descripcion) {  
        this.nivel = nivel;  
        this.descripcion = descripcion;  
    }  
  
    public int getNivel() {  
        return nivel;  
    }  
  
    public String getDescripcion() {  
        return descripcion;  
    }  
}
```

6. MÉTODOS PRINCIPALES DE ENUM

Explicación de los principales métodos, Especificación de 6 métodos entre los propios de Enum y los generados/relacionados al trabajar con enumerados.

6.1 values()

Devuelve un array con todas las constantes del enumerado, en el orden en que fueron declaradas.

```
for (Prioridad p : Prioridad.values()) {  
    System.out.println(p);  
}
```

Uso habitual: recorrer todas las opciones, crear menús, listar datos permitidos.

6.2 valueOf(String name)

Convierte un texto exacto en la constante correspondiente.

```
Prioridad p = Enum.valueOf(Prioridad.class, "ALTA");
```

Si el texto no coincide exactamente, lanza `IllegalArgumentException`. Ojo!!(no acepta variaciones como "alta" o "Alta "sin tratarlas antes).

6.3 name()

Devuelve el nombre exacto de la constante tal como fue declarado.

```
System.out.println(Prioridad.ALTA.name());
```

Útil para: persistencia simple, depuración, comparaciones exactas.

6.4 ordinal()

Devuelve la posición de la constante dentro del enum, empezando por 0.

```
System.out.println(Prioridad.BAJA.ordinal());
```

Importante!!: no conviene usar `ordinal()` para lógica de negocio ni para guardar datos, porque si cambias el orden de declaración, el valor cambia.

6.5 toString

Puede usarse para obtener una representación en texto. Por defecto suele coincidir con el nombre, pero se puede sobrescribir.

```
@Override
public String toString() {
    return descripcion;
}
```

Permite mostrar textos legibles al usuario sin perder la constante interna.

6.6 compareTo(E o)

Compara dos constantes del mismo enum según su orden de declaración.

```
System.out.println(Prioridad.BAJA.compareTo(Prioridad.ALTA));
```

Si la primera aparece antes, el resultado será negativo.

6.7 getDeclaringClass()

Devuelve la clase enum a la que pertenece la constante.

```
System.out.println(Prioridad.ALTA.getDeclaringClass());
```

6.8 equals(Object other)

Compara si dos referencias apuntan a la misma constante. En enums también se suele usar ==, que es seguro y recomendable.

```
return enumValue.equals(Enum.valueOf(Tipo.class, "ESPECIAL"));
```

6.9 Enum.valueOf(Class<T> enumClass, String name)

Versión genérica del método de conversión por nombre

```
Dia dia = Enum.valueOf(Dia.class, "LUNES")
```

7. BUENAS PRÁCTICAS AL USAR ENUM

- usar enum cuando el conjunto de valores sea cerrado
- añadir atributos si cada constante necesita datos extra
- sobrescribir toString() si quieres mostrar texto legible
- usar == para comparar enums
- usar values() para menús y listados
- validar bien la entrada del usuario antes de usar valueOf()

Cosas a evitar:

- usar ordinal() como si fuera un identificador estable
- meter demasiada lógica compleja dentro del enum

8. DESVENTAJAS O LIMITACIONES DE ENUM

- el conjunto de valores es fijo en compilación
- no sirve si los valores deben venir de base de datos o cambiar dinámicamente
- valueOf() es estricto y puede fallar fácilmente con entradas del usuario
- un enum muy grande con demasiada lógica puede volverse poco mantenible

9. EJEMPLOS DE USO REALES

enum en casos de aplicaciones reales:

- estados de pedidos
- categorías de productos
- nivel de acceso
- tipos de incidencias
- métodos de pago
- días de la semana
- meses
- estados de una reserva

10. CONCLUSIÓN

La clase especial enum es una herramienta muy importante en Java para modelar conjuntos cerrados de valores de forma segura, clara y orientada a objetos. Frente al uso antiguo de enteros o cadenas, mejora la legibilidad, evita errores y permite añadir comportamiento. En Java 21 sigue siendo totalmente actual y muy recomendable en programas bien diseñados.

11. EJEMPLO HECHO EN ECLIPSE(CARPETA)

12. POSIBLES DIFICULTADES

confundir enum con constantes simples Mucha gente piensa que un enum solo sustituye a varias constantes, pero en realidad puede tener comportamiento y datos propios.

valueOf() falla fácilmente Si el usuario escribe alta en minúsculas y el enum es ALTA, fallará. Por eso suele ser mejor crear métodos auxiliares como desdeNumero() o normalizar cadenas.

usar ordinal() incorrectamente Aunque existe, no es buena idea usarlo como identificador persistente.

creer que puede cambiar dinámicamente Un enum es fijo. Si necesitas categorías que se cargan desde fichero o base de datos, un enum ya no siempre es la mejor opción.

13. COMPARACIÓN BREVE CON VERSIONES ANTERIORES

A pesar de que los enum se introdujeron en Java 5, su implementación ha progresado y continúa siendo un elemento esencial en ultimas versiones Java. Lo importante es explicar que:

- antes se usaban más enteros y cadenas
- con enum se ganó seguridad de tipos
- en Java moderno se integra mejor con switch
- en Java 21 sigue siendo totalmente vigente y recomendado

